Performance Analysis of LLM Inference Engines

PG60301 Rodrigo Domingues
MEI – High Performance Data Analysis
University of Minho
Email: pg60301@alunos.uminho.pt

PG61539 Rafael Correia
MEI – High Performance Data Analysis
University of Minho
Email: pg61539@alunos.uminho.pt

***Abstract*—This work presents a detailed performance analysis of LLM inference on CPU-based systems using the llama.cpp framework, following the Track A1 (Single-Engine Deep Dive) methodology. Three quantised models—Gemma, Meta LLaMA, and TinyLlama—are evaluated under controlled conditions to characterise their performance across different workloads.**

To ensure measurement stability, each experiment includes a warm-up phase consisting of preliminary inference requests to preload model weights into memory. The evaluation explores a grid of configurations defined by varying the number of workers (4, 8, 12) and threads (10, 20, 30, 40). For each configuration, performance metrics are collected across three prompt categories—small, medium, and large—representing increasing input sizes. Each prompt category is executed three times, resulting in nine measurements per configuration, which are stored in structured log files (e.g., W4T10.txt).

The collected data is aggregated by computing the mean performance for each prompt size, enabling consistent comparison and visualisation across configurations. Metrics such as Time to First Token (TTFT), Time Per Output Token (TPOT) and throughput are analysed to assess the impact of parallelism and workload characteristics.

This study provides a systematic and hardware-aware evaluation of CPU-based LLM inference, highlighting how execution parameters and workload properties influence performance and scalability.

***Index Terms*—LLM inference, llama.cpp, CPU performance, benchmarking, TTFT, TPOT, throughput, prompt size analysis.**

I. Introduction

Understanding the performance characteristics of large language model (LLM) inference systems is essential for efficient deployment, particularly in CPU-only environments where hardware resources are limited. This project focuses on the analysis of LLM inference performance using the *Single-Engine Deep Dive* approach, where a single inference engine is explored across multiple configurations to identify performance trends and bottlenecks.

In this work, the `llama.cpp` inference engine is selected due to its strong support for CPU-based execution and flexible configuration options. Three quantised models—Gemma, Meta LLaMA, and TinyLlama—are evaluated under controlled conditions, all using the same quantisation format (Q4_K_M) to ensure consistency and comparability across experiments. The analysis systematically explores the impact of varying the number of workers and threads, as well as different prompt sizes, on key performance metrics such as Time to First Token (TTFT), Time Per Output Token (TPOT), and throughput.

Experimental results show that increasing parallelism does not necessarily lead to performance improvements. In particular, higher numbers of workers and threads often result in diminishing returns or even degraded performance, suggesting the presence of contention and resource saturation. Additionally, a clear distinction is observed between the prefill and decode phases, where TTFT is strongly influenced by prompt size, while TPOT remains relatively stable and is primarily constrained by hardware limitations.

Overall, this study provides a systematic evaluation of how configuration parameters affect LLM inference performance on CPU systems, highlighting key trade-offs and offering insights into efficient deployment strategies.

II. Methodology

All experiments were conducted on ARM nodes of Deucalion. The inference engine used in this study is `llama.cpp`, selected due to its efficient support for CPU-based execution and flexible configuration options. Three models were evaluated: Meta LLaMA, TinyLlama, and Gemma, all using the Q4_K_M quantisation format to ensure consistency in memory footprint and computational behaviour across experiments.

The experimental design explores a grid of configurations defined by the number of workers $\{4, 8, 12\}$ and threads $\{10, 20, 30, 40\}$. For each configuration, inference is executed using three prompt categories: small, medium, and large. Each prompt category is executed three times per configuration in order to reduce variance and ensure statistical stability of the results.

The evaluation uses three standardized prompts designed to represent increasing workload complexity. The small prompt asks a concise factual question: "What is the largest planet in the solar system, and how does its size compare to Earth? Provide a brief explanation including its composition and any notable features such as its atmosphere or magnetic field." The medium prompt focuses on conceptual understanding and comparative reasoning in machine learning, covering supervised, unsupervised, and reinforcement learning, and requesting examples and use cases for each paradigm. The large prompt provides a long-form technical context on transformer-based language models, CPU execution constraints, memory bandwidth limitations, and quantization, followed by a question on the challenges of CPU inference and the trade-offs introduced by quantization. Together, these prompts are

intended to stress different stages of the inference pipeline, from short factual generation to long-context reasoning and memory-intensive processing.

Prior to measurement, a warm-up phase is performed using short conversational prompts (e.g., greetings) to preload model weights into memory and mitigate cold-cache effects.

For each configuration, performance data is stored in structured log files named according to the corresponding worker-thread combination (e.g., W4T10.txt). Each file contains nine measurements, corresponding to the three prompt categories and their three repetitions. The recorded metrics include Time to First Token (TTFT), Time Per Output Token (TPOT), prefill throughput, decode throughput, and total execution time.

A Python-based analysis script is used to automate the post-processing stage of the experiments. The script parses the structured log files generated for each worker-thread configuration, extracts the recorded metrics, and computes the average values across repeated runs for each prompt category. In addition to data aggregation, the script generates the tables and plots used throughout the report, providing a consistent and reproducible workflow for performance analysis.

The resulting aggregated measurements enable a systematic evaluation of how different configurations affect inference behaviour across varying workload sizes. These processed results are subsequently used to analyse performance trends, identify scalability limitations, and compare the impact of workers, threads, and prompt complexity on the observed metrics.

III. Results

The complete aggregated measurement tables for all evaluated models are provided in the appendix section. This section focuses primarily on the graphical analysis and discussion of the most relevant performance trends observed across the different configurations.

A. Meta LLaMA Results

1) Prefill and Decode Throughput: Figures 1 and 2 present the prefill and decode throughput behaviour of the Meta LLaMA model across different worker-thread configurations and prompt sizes.

Overall, the results show that throughput remains relatively stable as the number of threads increases from 10 to 40, indicating limited scalability from additional thread-level parallelism. For decode throughput, small prompts consistently achieve the highest throughput values, reaching approximately 7.2 tokens/s for configurations using 4 workers, while medium and large prompts exhibit progressively lower throughput values. As the number of workers increases from 4 to 12, a gradual decrease in decode throughput is observed across all prompt categories, suggesting increased contention for shared CPU resources such as caches and memory bandwidth.

A similar trend is visible for prefill throughput. Small prompts maintain the highest throughput values, close to 9.7 tokens/s across all configurations, while medium and large prompts operate at lower throughput levels, typically between 7.4 and 7.7 tokens/s. Unlike decode throughput, prefill throughput shows only minor fluctuations across thread configurations, indicating that increasing the number of threads provides limited performance benefit during prompt processing.

The relatively small variation between thread configurations suggests that the inference workload is not strongly compute-bound under the evaluated conditions. Instead, the results indicate that memory access behaviour and shared-resource contention likely dominate performance, limiting the effectiveness of additional parallelism on CPU-only execution.

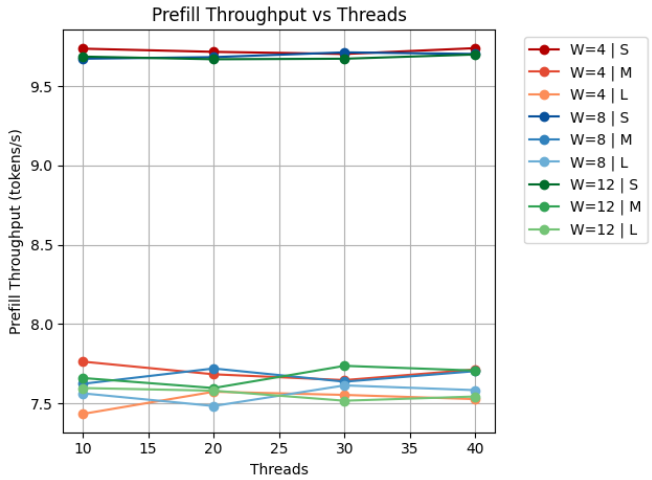

Fig. 1. Prefill throughput as a function of the number of threads for different worker configurations and prompt sizes using the Meta LLaMA model.

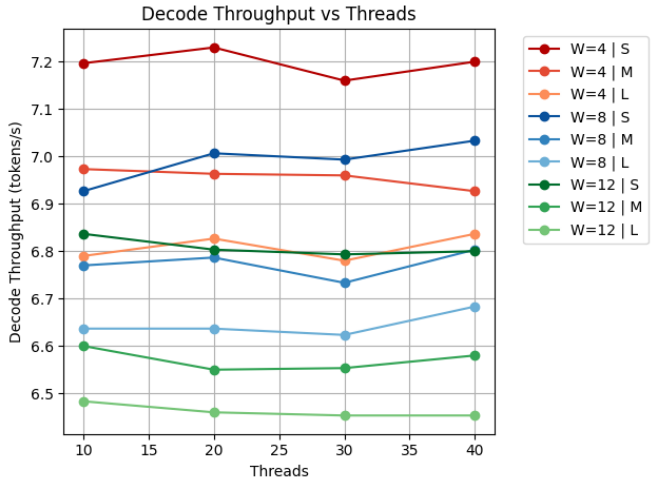

Fig. 2. Decode throughput as a function of the number of threads for different worker configurations and prompt sizes using the Meta LLaMA model.

Figure 3 presents the ratio between prefill and decode throughput for the evaluated Meta LLaMA configurations. This ratio provides insight into the relative efficiency of prompt processing compared to token generation.

Across all configurations, the prefill stage consistently achieves higher throughput than the decode stage. The highest

ratios are observed for small prompts, ranging approximately between 1.34 and 1.43, indicating that prompt processing is significantly more efficient than sequential token generation for smaller workloads. In contrast, medium and large prompts exhibit lower ratios, typically between 1.09 and 1.18, suggesting a more balanced cost between prefill and decode phases as prompt complexity increases.

Additionally, the ratio tends to increase slightly as the number of workers grows from 4 to 12, particularly for small prompts. This behaviour indicates that decode throughput degrades more noticeably than prefill throughput under higher concurrency levels, reinforcing the hypothesis that decoding is more sensitive to shared-resource contention and memory access limitations.

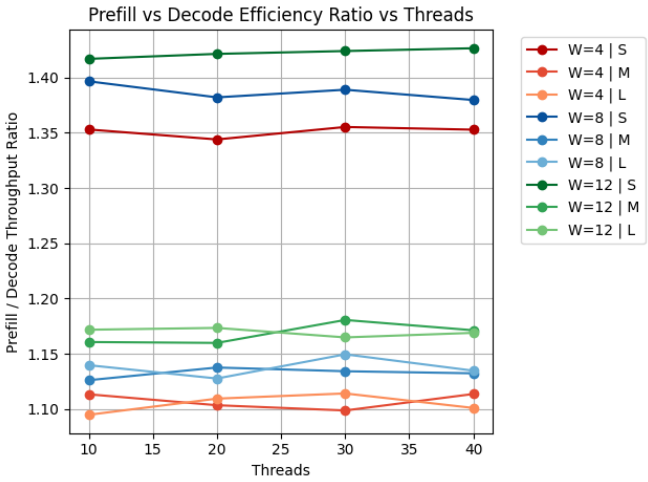

Fig. 3. Ratio between prefill and decode throughput as a function of the number of threads for different worker configurations and prompt sizes using the Meta LLaMA model.

2) Time to First Token vs Threads: Figure 4 presents the TTFT behaviour of the Meta LLaMA model across all prompt sizes (small, medium, and large) and worker-thread configurations.

For small prompts, TTFT remains highly stable across all configurations, with values around 5.1 seconds, indicating that thread-level parallelism has negligible impact on prefill latency for short inputs. Medium prompts exhibit significantly higher TTFT values, approximately 25 seconds, while large prompts reach values in the range of 68–70 seconds, reflecting the strong dependence of TTFT on input sequence length.

Across all prompt sizes, variations in the number of workers and threads do not produce meaningful changes in TTFT. The observed fluctuations remain within a narrow range consistent with measurement variability, suggesting that TTFT is primarily dominated by memory access and prompt processing costs rather than compute parallelism.

3) Time to First Token vs Prompt Size: Figure 5 presents the relationship between TTFT and prompt size aggregated across all worker-thread configurations for the Meta LLaMA model.

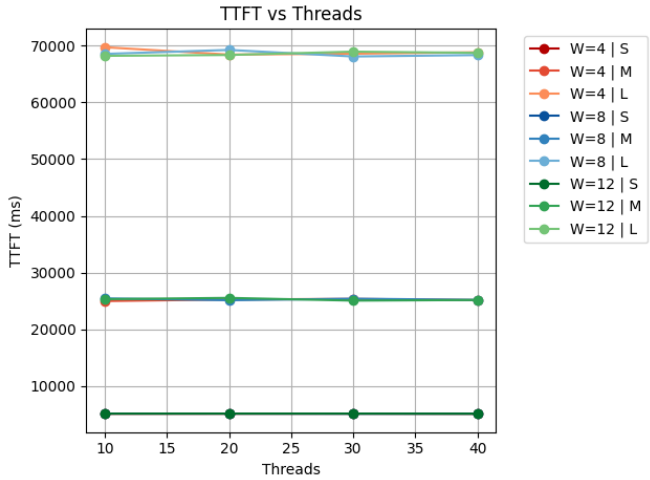

Fig. 4. TTFT as a function of the number of threads for all prompt sizes and worker configurations using the Meta LLaMA model.

The results clearly show a strong dependence of TTFT on prompt length. Small prompts consistently exhibit the lowest TTFT values, clustered around 5.1 seconds, with very low variance across repeated measurements. Medium prompts increase TTFT significantly to approximately 25 seconds, while large prompts reach values in the range of 68–70 seconds. This scaling behaviour is consistent across all configurations, indicating that prompt size is the dominant factor influencing prefill latency.

Despite the large number of configurations included in the dataset, no meaningful separation is observed between different worker or thread settings within each prompt category. The spread of values within each group remains relatively narrow and is consistent with expected measurement variability. This reinforces the conclusion that TTFT is primarily driven by input length and memory access cost, rather than parallelism settings in the evaluated CPU-based inference setup.

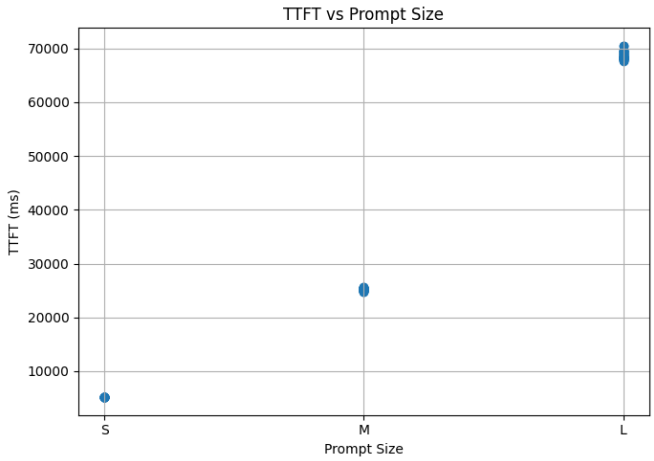

Fig. 5. TTFT as a function of prompt size aggregated across all worker and thread configurations for the Meta LLaMA model.

4) Time Per Output Token (TPOT): Figure 6 presents the TPOT behaviour of the Meta LLaMA model as a function of the number of threads across all prompt sizes and worker configurations.

Overall, TPOT remains highly stable across the evaluated configurations, with only minor variations observed between different thread counts. Small prompts exhibit TPOT values around 138–145 ms/token, while medium prompts increase slightly to approximately 143–152 ms/token. Large prompts consistently show the highest latency, typically between 146–155 ms/token.

Despite varying the number of threads from 10 to 40, no consistent performance improvement is observed. The small fluctuations in measured values remain within expected experimental variability, indicating that the decoding phase is largely insensitive to thread-level parallelism. This suggests that TPOT is primarily constrained by memory bandwidth and shared-resource contention rather than compute availability.

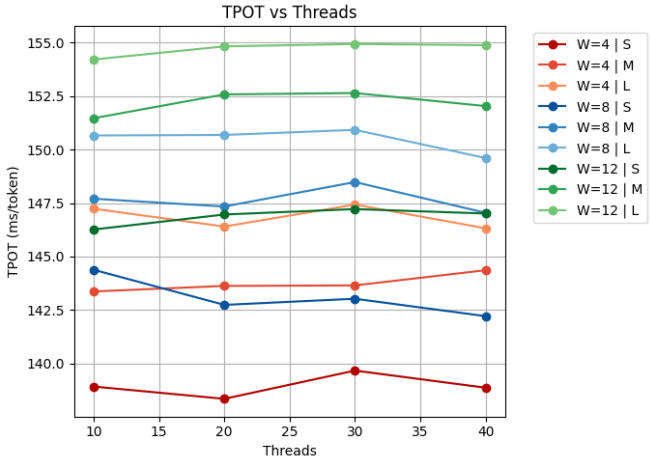

Fig. 6. TPOT as a function of the number of threads for all prompt sizes and worker configurations using the Meta LLaMA model.

5) TPOT vs Prompt Size: The TPOT metric is further analysed with respect to prompt size in order to evaluate whether input length has a measurable impact on per-token decoding efficiency. Unlike TTFT, which is strongly affected by prompt size due to the initial processing phase, TPOT reflects the steady-state decoding behaviour of the model.

Figure 7 presents the aggregated TPOT values across all configurations as a function of prompt size. The results show a clear and consistent increase in TPOT from small to medium and large prompts. However, this variation remains relatively moderate and follows a smooth trend, indicating that the per-token generation cost is only weakly affected by prompt length under Q4_K_M quantisation.

Specifically, small prompts exhibit the lowest TPOT values, while medium and large prompts show progressively higher latency per token. This behaviour suggests that longer contexts introduce additional overhead in attention and KV-cache access patterns, although the effect is not dominant enough to cause instability or large deviations.

Overall, the results indicate that TPOT is primarily stable across workloads, with prompt size introducing a gradual but limited increase in decoding cost per token.

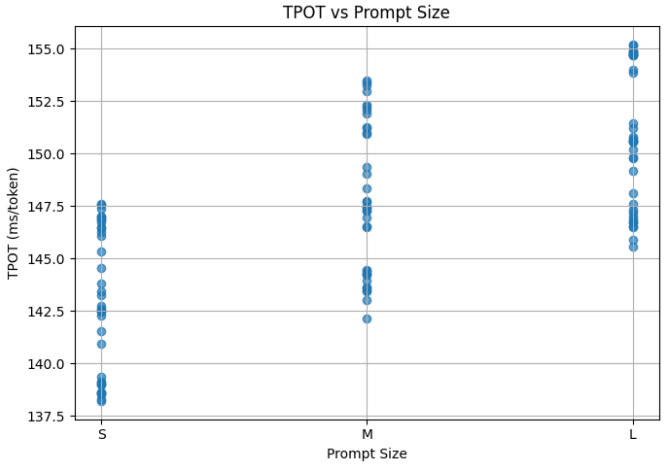

Fig. 7. TPOT (ms/token) as a function of prompt size aggregated across all worker-thread configurations. The results show a gradual increase in decoding latency per token from small to large prompts, indicating a mild dependence of TPOT on input length.

6) Total Execution Time vs Total Tokens: The relationship between total execution time and the number of processed tokens is analysed to evaluate the overall efficiency of the inference pipeline, including both prompt processing and token generation stages.

Figure 8 illustrates the correlation between total tokens (prompt + generated tokens) and total execution time across all configurations and prompt sizes. The results show an approximately linear scaling trend, indicating that execution time increases proportionally with the number of processed tokens.

Across prompt sizes, clear differences in performance regimes can be observed. Small prompts consistently exhibit the lowest execution times, forming a compact and stable cluster. Medium prompts operate in a higher latency regime and show the largest dispersion in values, indicating higher sensitivity to workload variability and system scheduling effects. In contrast, large prompts also operate in a high-latency regime but tend to concentrate within a narrower region, suggesting more stable execution behaviour despite increased computational load.

Overall, these results confirm that total execution time is strongly driven by token count, while also highlighting that prompt size introduces distinct performance regimes with different levels of variability.

B. TinyLLaMA Results

This section presents the performance evaluation of TinyLLaMA across different system configurations, focusing on decode throughput, prefill throughput, and their relative efficiency. The analysis is structured around three main metrics, each illustrated through aggregated plots over worker counts, thread configurations, and prompt sizes.

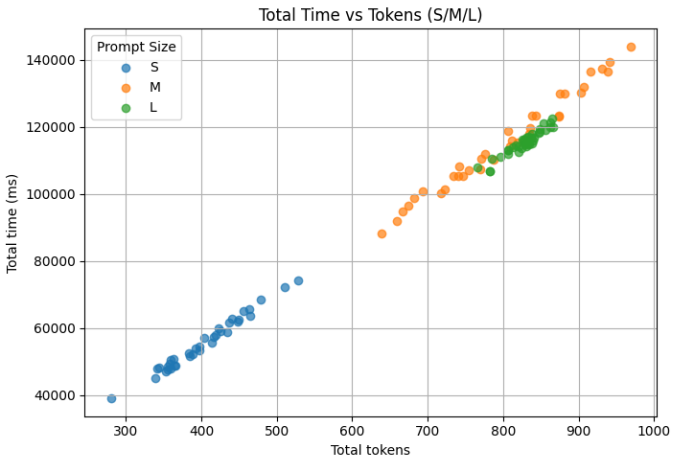

Fig. 8. Total execution time as a function of total processed tokens across all configurations and prompt sizes. Small prompts exhibit consistently lower execution times, while medium and large prompts operate in higher latency regimes, with medium showing greater dispersion and large concentrating within a narrower range.

1) Prefill and Decode Throughput vs Threads: Figure 9 and Figure 10 present the evolution of prefill and decode throughput as a function of the number of threads for different worker configurations and prompt sizes.

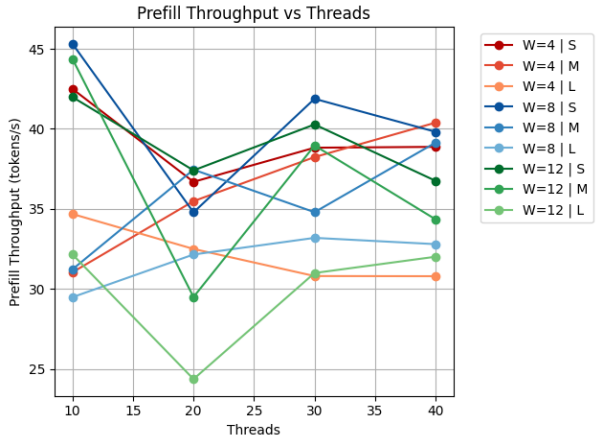

Fig. 9. TinyLLaMA prefill throughput vs number of threads across worker and prompt configurations.

Unlike the behavior previously observed for Meta LLaMA, TinyLLaMA exhibits substantially higher throughput variability across nearly all evaluated configurations. Both prefill and decode throughput measurements present significant oscillations as the number of threads changes, with several configurations showing abrupt increases and decreases in tokens/s rather than maintaining a stable scaling tendency.

This behavior is particularly visible in the prefill phase, where throughput values fluctuate aggressively between consecutive thread counts. For example, several configurations alternate between throughput gains and losses of multiple tokens/s as thread counts increase, suggesting that TinyLLaMA

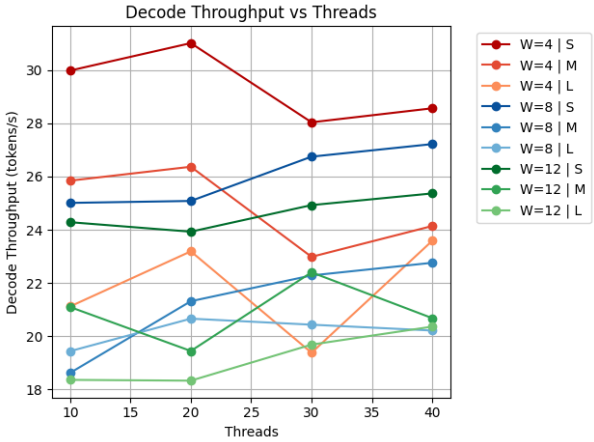

Fig. 10. TinyLLaMA decode throughput vs number of threads across worker and prompt configurations.

is considerably more sensitive to scheduling overhead, cache contention, and workload distribution effects.

Decode throughput also demonstrates noticeable instability compared to Meta LLaMA. Although decode values remain within narrower ranges than prefill throughput, the model still presents recurrent throughput jumps and drops across thread configurations, especially for medium and large prompts. The absence of a smooth scaling pattern indicates that performance is heavily dependent on transient runtime conditions rather than purely on additional parallel resources.

Overall, TinyLLaMA demonstrates a considerably less stable throughput profile than Meta LLaMA, with higher sensitivity to both thread count and worker configuration across all prompt categories.

2) Prefill-to-Decode Throughput Ratio: Figure 11 presents the ratio between prefill and decode throughput for the evaluated configurations.

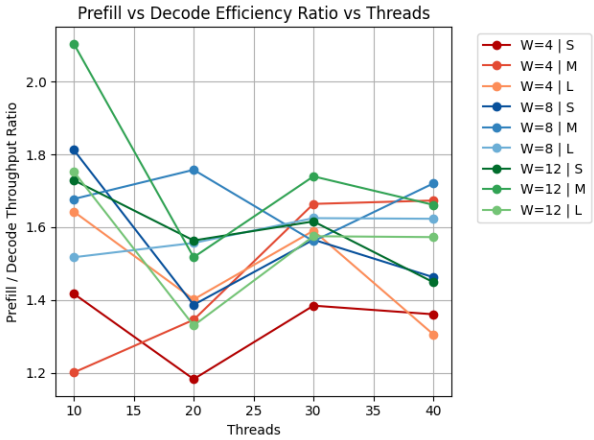

Fig. 11. Ratio between prefill and decode throughput vs number of threads.

The prefill-to-decode throughput ratio further highlights the unstable performance characteristics of TinyLLaMA. In contrast to Meta LLaMA, where the ratio remained relatively

stable across configurations, TinyLLaMA presents substantial fluctuations between consecutive thread counts and worker configurations.

Several configurations exhibit large jumps in ratio values, frequently alternating between significantly different efficiency regimes as thread counts increase. This behavior is especially pronounced for medium and large prompts, where the ratio can shift considerably even under small configuration changes. Such variability indicates that the balance between prefill and decode efficiency is not consistently maintained throughout execution.

The observed oscillations suggest that TinyLLaMA reacts more aggressively to runtime resource contention and scheduling effects, producing less predictable throughput relationships between execution phases. Consequently, the ratio plots reinforce the conclusion that TinyLLaMA exhibits substantially more volatile performance behavior compared to Meta LLaMA.

3) Time to First Token vs Threads: Figure 12 presents the evolution of TTFT across different thread and worker configurations for all prompt sizes.

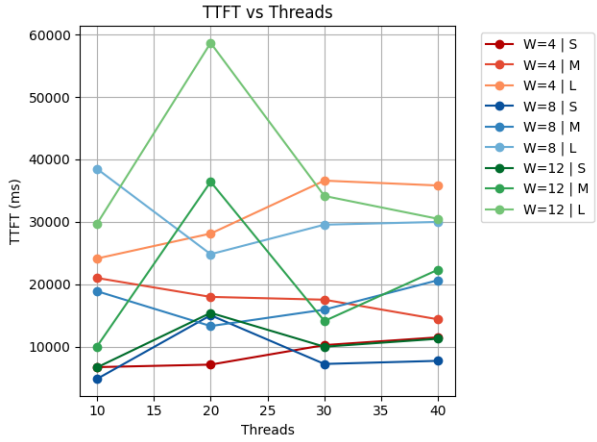

Fig. 12. TinyLLaMA TTFT vs number of threads across worker and prompt configurations.

The TTFT measurements reveal a substantially more unstable behavior compared to Meta LLaMA, with large oscillations observed across nearly all evaluated configurations. Rather than following a predictable scaling tendency, TinyLLaMA frequently exhibits abrupt latency spikes when modifying the number of threads or workers.

This instability is especially visible for medium and large prompts, where consecutive thread configurations often present substantial TTFT variations. In several cases, increasing the thread count from the initial 10-thread configuration results in latency differences of approximately $\pm 25\%$, indicating strong sensitivity to runtime scheduling effects, memory contention, and synchronization overhead.

A particularly notable case is the configuration using 12 workers and 20 threads (W=12, T=20), which behaves as a clear outlier for both medium and large prompts. In this configuration, TTFT reaches approximately 36 s for medium prompts and nearly 59 s for large prompts, significantly exceeding neighboring measurements. This anomalous behavior is later reinforced in Figure 13, where these executions appear among the highest latency values observed across all configurations.

Despite the presence of outliers and high variability, the results still indicate that increasing the number of workers or threads does not provide consistent TTFT improvements. Across most configurations, changes in parallelism mainly introduce instability rather than measurable latency reduction.

Small prompts also demonstrate irregular behavior, although with lower absolute latency values. Even within the smallest workload category, TTFT can fluctuate significantly between configurations, indicating that the model does not maintain a stable initialization cost under changing parallel execution conditions.

Overall, TinyLLaMA exhibits highly volatile TTFT characteristics, contrasting sharply with the smoother and more stable latency behavior observed for Meta LLaMA.

4) Time to First Token vs Prompt Size: Figure 13 presents the relationship between TTFT and prompt size across all evaluated configurations.

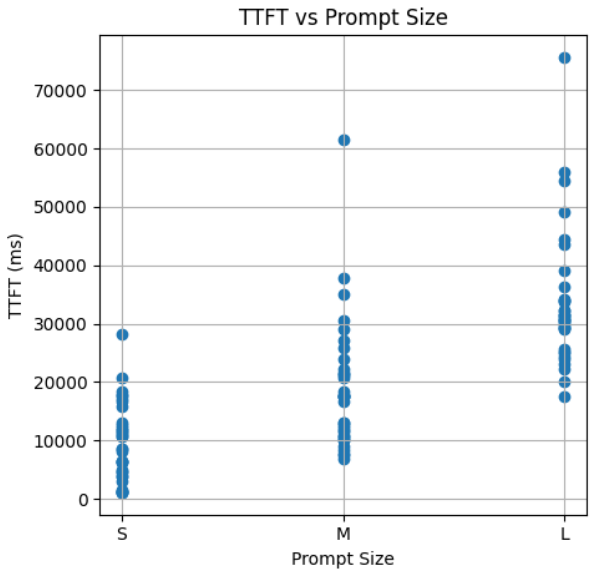

Fig. 13. TinyLLaMA TTFT distribution across prompt sizes.

The distribution confirms a clear increase in TTFT as prompt size grows, with small prompts consistently occupying the lowest latency region and large prompts producing the highest execution times. However, unlike Meta LLaMA, TinyLLaMA exhibits a substantially wider spread of values within each prompt category.

Medium prompts display particularly high variance, with executions distributed across a broad latency range and several configurations generating large outlier values. Large prompts also demonstrate significant variability, although their measurements tend to concentrate around higher latency regions with somewhat more consistent clustering behavior.

Although the overall trend still follows the expected progression from small to medium to large prompts, the observed distributions present considerable overlap between categories. In particular, the worst-performing small-prompt executions achieve TTFT values comparable to the best-performing large-prompt executions. This overlap further highlights the instability of the model under different runtime configurations.

The results further reinforce the conclusion that TinyLLaMA presents unstable latency characteristics under varying execution conditions, with prompt size amplifying the observed performance volatility.

5) Time Per Output Token vs Threads: Figure 14 presents the TPOT behavior of TinyLLaMA across different worker-thread configurations and prompt sizes.

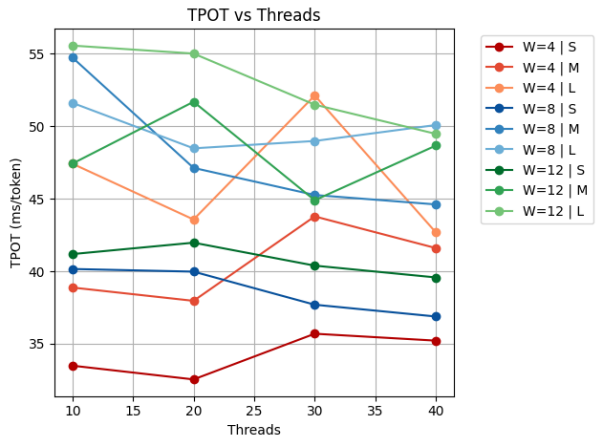

Fig. 14. TinyLLaMA TPOT vs number of threads across worker and prompt configurations.

Compared to TTFT, TPOT exhibits a slightly more stable behavior across configurations, although noticeable variability is still present throughout the evaluated thread counts and worker settings. Small prompts consistently achieve the lowest decoding latency, typically remaining within the 30–42 ms/token range, while medium and large prompts operate at progressively higher TPOT values.

For several configurations, changing the number of threads produces fluctuations of approximately 10–20% relative to the initial 10-thread measurements, particularly for medium and large prompts. However, unlike TTFT, the observed oscillations remain comparatively moderate and do not produce extreme latency spikes.

Overall, the decoding phase appears more stable than the prefill phase, although the model continues to demonstrate sensitivity to runtime contention and scheduling effects under different execution configurations.

6) Time Per Output Token vs Prompt Size: Figure 15 presents the TPOT distribution across prompt sizes for all evaluated configurations.

The results show a gradual increase in TPOT as prompt size grows, with small prompts occupying the lowest latency region and large prompts generally producing the highest

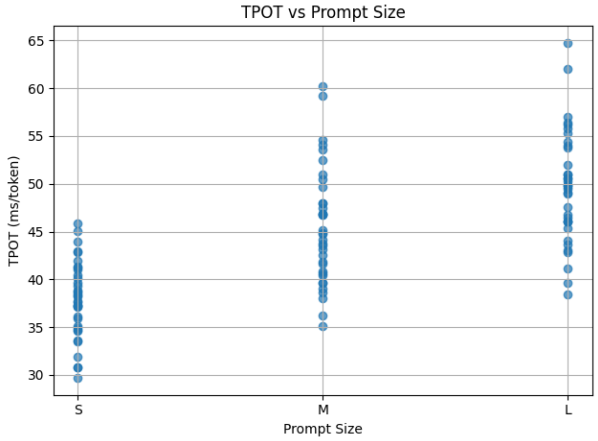

Fig. 15. TinyLLaMA TPOT distribution across prompt sizes.

decoding costs per token. However, unlike the Meta LLaMA TPOT distribution, where distinct measurement clusters could be observed within each prompt category corresponding to different worker configurations (W=4, 8, and 12), TinyLLaMA does not exhibit such separation.

Instead, the measurements inside each prompt-size category appear significantly more blended together, with no clear grouping associated with specific worker configurations. This behavior indicates that the impact of worker-level parallelism becomes less predictable, producing overlapping execution regimes across configurations.

Medium and large prompts still present higher variance and broader distributions, while small prompts remain comparatively more concentrated. Nevertheless, the absence of clearly separated clusters further reinforces the unstable and inconsistent decoding behavior of TinyLLaMA under different execution conditions.

Despite this variability, TPOT maintains a more stable overall trend than TTFT, suggesting that the decode phase remains less sensitive to configuration changes than the prefill stage.

7) Total Execution Time vs Total Tokens: The relationship between total execution time and the number of processed tokens is analysed to evaluate the overall efficiency of the inference pipeline, including both prompt processing and token generation stages.

Figure 16 shows a clear positive correlation between total processed tokens and total execution time, indicating that larger workloads generally require longer execution times. However, unlike the behavior previously observed for Meta LLaMA, the resulting distributions are considerably more dispersed and less structured.

For Meta LLaMA, prompt categories formed relatively compact regions with visible separation between execution regimes. In contrast, TinyLLaMA presents substantial overlap between prompt sizes, with several medium-prompt executions reaching execution times comparable to large prompts, and some large-prompt executions appearing close to the upper

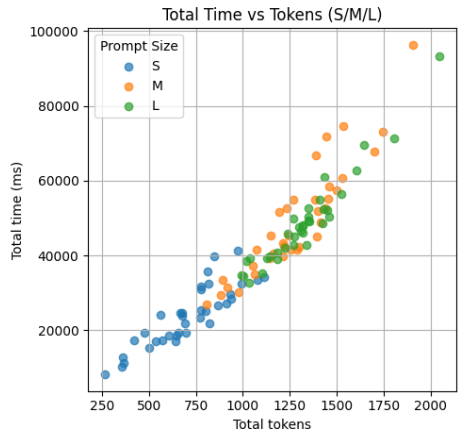

Fig. 16. TinyLLaMA total execution time as a function of total processed tokens across all prompt sizes and configurations.

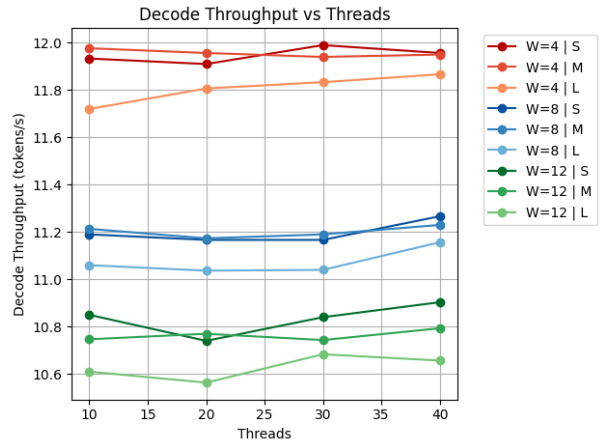

Fig. 18. Gemma decode throughput vs number of threads across worker and prompt configurations.

range of medium workloads.

The measurements also exhibit significantly higher dispersion for similar token counts, indicating that runtime conditions and configuration variability strongly affect execution behavior. In particular, medium prompts present the widest spread of values, reinforcing the unstable performance characteristics already observed in TTFT and TPOT measurements.

Overall, although total execution time still scales approximately with token count, TinyLLaMA demonstrates a considerably less predictable execution profile than Meta LLaMA, with weaker clustering and higher variability across configurations.

C. Gemma Results

1) Prefill and Decode Throughput vs Threads: Figures 17 and 18 present the prefill and decode throughput behavior of the Gemma model across different worker-thread configurations and prompt sizes.

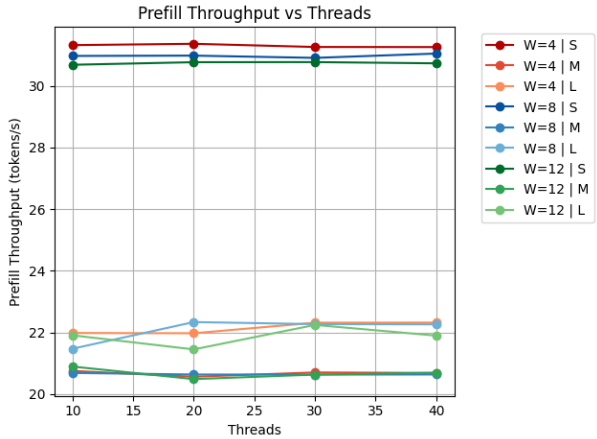

Fig. 17. Gemma prefill throughput vs number of threads across worker and prompt configurations.

Unlike TinyLLaMA, Gemma exhibits highly stable throughput behavior across all evaluated configurations. Both prefill and decode throughput remain nearly constant as the number of threads increases from 10 to 40, with only very small fluctuations observed between consecutive measurements.

Decode throughput shows a slight decrease as the number of workers increases from 4 to 12. Configurations using 4 workers consistently achieve the highest throughput values, operating close to 12 tok/s, while configurations using 12 workers remain around 10.6–10.9 tok/s. This behavior suggests a moderate degradation caused by increased resource contention under higher concurrency levels.

A similar trend is observed for prefill throughput. Small prompts consistently achieve the highest throughput values, remaining close to 31 tok/s across all configurations, while medium and large prompts stabilize around 20–22 tok/s. As with decode throughput, increasing the number of threads produces almost no measurable improvement, indicating that the workload is already saturating the available CPU resources.

Overall, Gemma demonstrates the most stable throughput profile among the evaluated models, with minimal variance across configurations and very predictable scaling behavior under different prompt sizes and worker-thread combinations.

2) Prefill-to-Decode Throughput Ratio: Figure 19 presents the ratio between prefill and decode throughput for the Gemma model across different worker-thread configurations and prompt sizes.

The prefill-to-decode throughput ratio remains highly stable across all configurations, closely mirroring the stability already observed in the individual throughput metrics. Across thread counts from 10 to 40, the ratio shows negligible variation, indicating that the relative efficiency between prefill and decode stages is largely invariant to parallelism settings.

For small prompts, the ratio consistently remains around 2.6–2.8 depending on the worker configuration, reflecting a substantially more efficient prefill stage compared to decoding. Medium prompts stabilize around 1.7–1.9, while large prompts

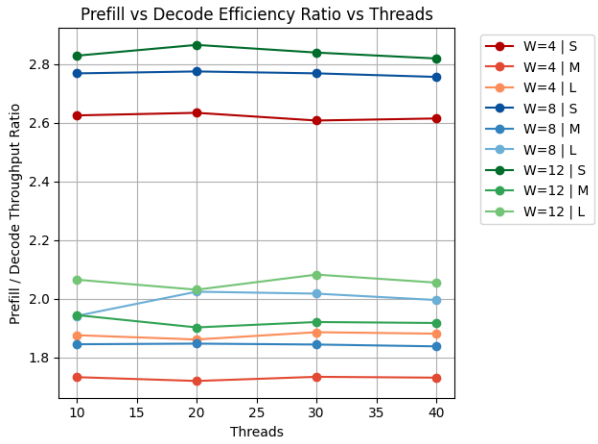

Fig. 19. Gemma prefill-to-decode throughput ratio vs number of threads across worker and prompt configurations.

typically remain close to 1.8–2.0. These differences indicate that prompt size has a stronger influence on the prefill/decode balance than thread-level scaling.

Unlike the behavior observed in TinyLLaMA, no significant oscillations or configuration-induced instability are present. Even when varying workers and threads, the ratio remains tightly clustered, suggesting that both execution stages scale proportionally under increased concurrency.

Overall, Gemma demonstrates a highly consistent prefill-to-decode relationship, reinforcing its characterization as the most stable model in terms of execution behavior among the evaluated systems.

3) Time to First Token vs Threads: Figure 20 presents the TTFT behavior of the Gemma model across different thread and worker configurations for all prompt sizes.

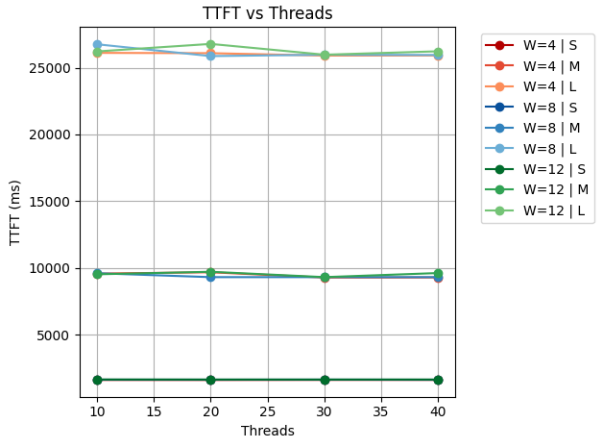

Fig. 20. Gemma TTFT vs number of threads across worker and prompt configurations.

Gemma exhibits highly stable TTFT behavior across all evaluated configurations, with only minimal fluctuations observed when varying the number of workers and threads. Unlike TinyLLaMA, no abrupt spikes or unstable execution regions are visible throughout the measurements.

The results show a clear dependence between TTFT and prompt size. Small prompts consistently remain around 1.6 s, medium prompts stabilize near 9.3–9.7 s, and large prompts operate close to 26 s across nearly all configurations. This behavior indicates that prompt size is the dominant factor influencing TTFT, while thread-level parallelism has negligible impact on prefill latency.

Across all worker configurations, increasing the number of threads from 10 to 40 produces only very small variations, typically within a narrow range consistent with expected measurement variability. Even configurations with higher worker counts maintain nearly identical latency profiles, suggesting that Gemma scales in a highly predictable manner under concurrent execution.

Overall, the results reinforce the strong execution stability already observed in throughput measurements, positioning Gemma as the most consistent model among the evaluated systems in terms of TTFT behavior.

4) Time to First Token vs Prompt Size: Figure 21 presents the relationship between TTFT and prompt size across all evaluated configurations.

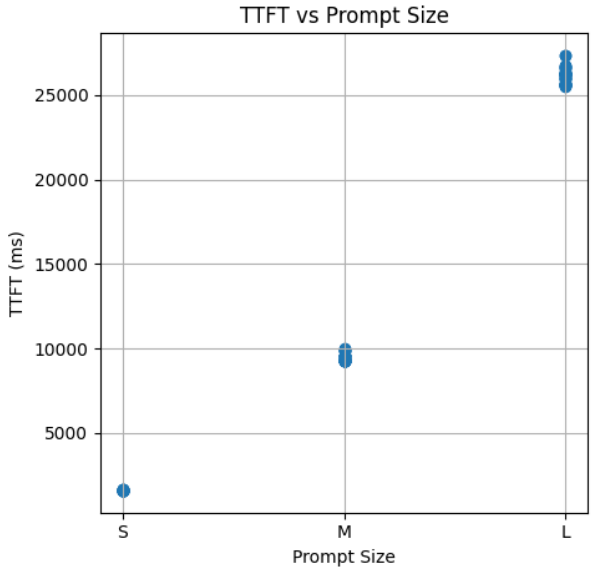

Fig. 21. Gemma TTFT distribution across prompt sizes.

The results show a clear and highly structured increase in TTFT as prompt size grows. Small prompts consistently occupy the lowest latency region around 1.6 s, medium prompts stabilize near 9–10 s, and large prompts remain concentrated around 26–27 s.

Unlike TinyLLaMA, the distributions exhibit very low variance and almost no overlap between prompt categories. Each prompt size forms a compact and clearly separated cluster, indicating highly predictable execution behavior across all worker-thread configurations.

The narrow spread of measurements within each category further demonstrates that runtime conditions and parallelism settings have minimal impact on TTFT for Gemma. Instead, prompt size remains the dominant factor influencing latency, producing smooth and consistent scaling between workload categories.

Overall, the TTFT distributions reinforce the strong execution stability already observed in the throughput analysis, confirming Gemma as the most stable and predictable model among the evaluated systems.

5) Time Per Output Token vs Threads: Figure 22 presents the TPOT behavior of the Gemma model across different worker-thread configurations and prompt sizes.

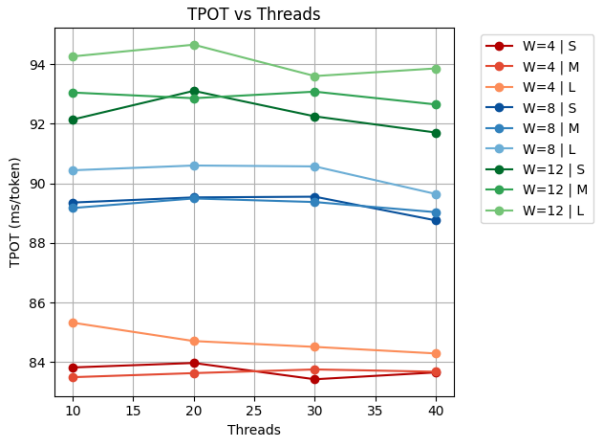

Fig. 22. Gemma TPOT vs number of threads across worker and prompt configurations.

Gemma exhibits highly stable TPOT behavior across all evaluated configurations, with only minimal variations observed when increasing the number of threads from 10 to 40. Unlike TinyLLaMA, no abrupt oscillations or unstable decoding regions are visible throughout the measurements.

For configurations using 4 workers, TPOT remains close to 83–85 ms/token across all prompt sizes. Increasing the number of workers introduces a gradual increase in decoding latency, with configurations using 8 workers stabilizing around 89–90 ms/token and configurations using 12 workers operating near 92–94 ms/token.

The impact of prompt size on TPOT is comparatively small. Large prompts consistently produce slightly higher latency values than small and medium prompts, but the differences remain modest and highly consistent across configurations.

Overall, the results indicate that thread-level parallelism provides almost no measurable improvement for decoding performance, while higher worker counts introduce a small but predictable degradation caused by increased resource contention. The extremely narrow spread of measurements further reinforces the strong execution stability demonstrated by Gemma throughout the evaluation.

6) Time per Output Token vs Prompt Size: Figure 23 presents the relationship between TPOT and prompt size across all evaluated configurations.

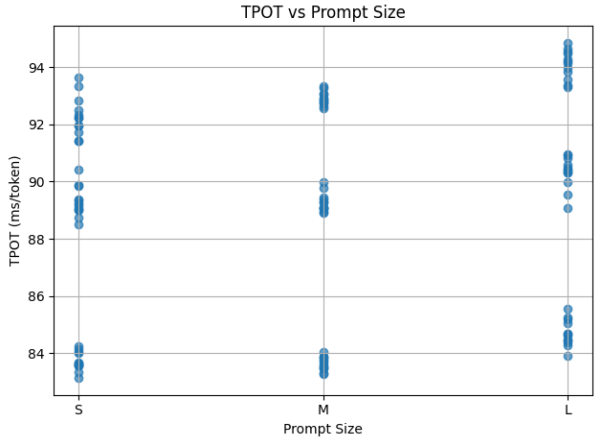

Fig. 23. Gemma TPOT distribution across prompt sizes.

The results show a small gradual increase in TPOT as prompt size grows, with small prompts consistently occupying the lowest latency region and large prompts producing the highest decoding costs per token. However, the separation between prompt categories remains relatively small, indicating that Gemma maintains highly stable decode performance across different workloads.

For each prompt size, the measurements form distinct clusters, most likely associated with the increase in worker count from $W = 4$ to $W = 8$ and $W = 12$. This clustered organization suggests that worker scaling introduces additional contention during data ingestion and decode scheduling, producing consistently separated TPOT regions even when prompt size remains unchanged.

Despite this separation between worker configurations, the variance inside each cluster remains extremely low, with only minor fluctuations between executions. Small, medium, and large prompts therefore preserve highly predictable decoding behavior, contrasting with the blended and unstable distributions previously observed in TinyLLaMA.

Overall, Gemma exhibits highly consistent TPOT characteristics, with stable scaling across prompt sizes and clearly structured execution behavior across worker configurations.

7) Total Time vs Total Tokens: Figure 24 presents the relationship between total execution time and the total number of generated tokens for all prompt sizes.

The results demonstrate a highly structured and predictable distribution across the three prompt categories. Small, medium, and large prompts form clearly separated clusters, with each category occupying its own distinct latency and token region. Unlike the behavior previously observed in TinyLLaMA, virtually no overlap exists between prompt groups.

Small prompts remain concentrated in the lowest execution-time region, medium prompts consistently occupy an intermediate range, and large prompts form a separate high-latency cluster associated with the largest token counts. The progression between categories follows a highly coherent scaling ten-

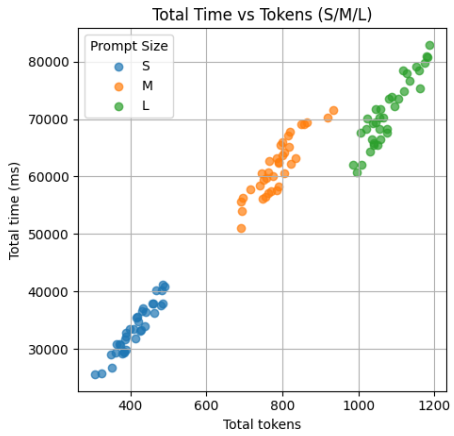

Fig. 24. Gemma total execution time as a function of total generated tokens.

dency, indicating that execution time increases proportionally with workload size.

Additionally, the measurements inside each cluster remain tightly grouped, showing only limited dispersion around the main trend line. This strong clustering behavior suggests that Gemma maintains highly stable execution dynamics under varying configurations, producing consistent latency characteristics for equivalent workloads.

Overall, the absence of significant overlap between prompt categories reinforces the idea that Gemma is a substantially more predictable model, exhibiting stable and reproducible performance across different execution scenarios.

IV. Discussion

A. Bottleneck Analysis

Across all evaluated models (TinyLlama, MetaLlama, and Gemma), inference behaviour is primarily governed by the standard two-phase autoregressive pipeline: a compute-heavy prefill stage and a memory-bandwidth-bound decode stage. However, the degree of stability, predictability, and clustering behaviour varies significantly between models.

TinyLlama exhibited the least stable behaviour across all metrics. While its smaller size resulted in relatively low absolute latency, its performance was highly sensitive to scheduling and concurrency effects. TTFT, TPOT, and throughput measurements showed significant overlap across prompt sizes and worker configurations. This indicates weak separation between execution regimes and suggests instability in KV-cache utilisation and batching efficiency. The model behaves as a system strongly influenced by scheduling contention rather than purely by computational scaling laws. Additionally, output generation was qualitatively unstable, with frequent hallucinations and weak instruction adherence.

MetaLlama showed substantially improved stability and clearer structural behaviour. TTFT and total-time metrics formed more distinct clusters across prompt sizes, with reduced overlap compared to TinyLlama. This indicates a more predictable scaling pattern in the prefill phase and better KV-cache consistency during decode. Throughput remained relatively stable across worker configurations, suggesting a saturated but well-behaved execution regime. Overall, MetaLlama operates closer to an ideal memory-bandwidth-bound inference system, with reduced sensitivity to scheduling noise. Output quality was also significantly improved, with concise and instruction-aligned responses and minimal hallucination.

Gemma demonstrated the most predictable and well-structured behaviour among all models. Performance metrics formed clean, non-overlapping clusters across all prompt sizes, indicating highly deterministic scaling behaviour. Each prompt category (small, medium, large) occupied a distinct region in the performance space, with minimal dispersion. This suggests stable KV-cache growth, efficient prefill execution, and well-controlled decode behaviour. Unlike the other models, increasing worker counts did not introduce significant noise, but rather shifted performance into clearly separated regimes, reflecting structured contention effects rather than instability.

B. Optimisations With the Highest Impact

Across all models, the most impactful optimisations are consistent and align with the dominant constraints of CPU-based autoregressive inference:

1) **Continuous batching**: improves CPU utilisation by reducing idle time and mitigating straggler effects in concurrent request execution.
2) **KV-cache optimisations**: reduce memory overhead and improve cache efficiency during decode, particularly under higher concurrency.
3) **Memory-efficient attention mechanisms**: reduce prefill cost by limiting memory traffic during attention computation.
4) **Length-aware scheduling**: improves batching efficiency by grouping similar sequence lengths and reducing padding waste.

These optimisations primarily target memory efficiency and scheduling behaviour rather than raw compute performance, which is consistent with inference workloads where memory bandwidth is the dominant constraint.

C. Limitations

The study is limited in several dimensions.

First, CPU usage and peak memory consumption were not measured. This restricts visibility into hardware-level bottlenecks and prevents precise attribution of saturation points in the inference pipeline.

Second, different quantization levels were not explored. As a result, the impact of reduced precision on latency, throughput, and stability could not be evaluated, limiting the analysis of performance–efficiency trade-offs in CPU inference settings.

V. Conclusion

A. Key Findings

The experimental evaluation across TinyLlama, MetaLlama, and Gemma demonstrates a clear relationship between model

choice, workload characteristics, and inference efficiency under CPU-based serving conditions. Performance is strongly structured across prompt sizes, with consistent separation between low, medium, and high workload regimes in all models, albeit with different degrees of clarity and stability.

Among the evaluated models, Gemma consistently achieves the best overall performance, presenting the lowest average latency and the most stable behaviour across all workload configurations. MetaLlama follows with intermediate performance, while TinyLlama exhibits higher variability and reduced stability under increased concurrency. These results indicate that more capable and better-optimised models provide not only lower latency but also more robust behaviour under varying load conditions.

B. System-Level Insights

The results highlight that inference performance is strongly dependent on the interaction between model characteristics and serving conditions. Differences in workload size and concurrency lead to clearly separated operational regimes, indicating that inference behaviour is highly structured rather than random.

Additionally, the observed results suggest that system-level factors such as scheduling behaviour and request concurrency significantly influence overall performance consistency. This reinforces the importance of considering both model properties and deployment conditions when evaluating inference systems.

C. Practical Implications

From a systems perspective, the findings indicate that model selection has a direct impact on both efficiency and stability in deployment scenarios. Larger and more capable models tend to offer better performance characteristics under identical serving conditions, particularly in terms of consistency across varying workloads.

Furthermore, the results suggest that inference systems benefit from workload-aware execution strategies, where request characteristics such as prompt size and concurrency are taken into account when managing execution resources.

D. Limitations

This study is subject to several limitations. Different quantization configurations were not evaluated, limiting the analysis of precision–performance trade-offs. Additionally, no measurements of CPU utilisation or peak memory consumption were collected, restricting visibility into underlying hardware resource usage and internal bottlenecks.

E. Final Remarks

Overall, the results demonstrate that inference performance in transformer-based language models is highly structured and dependent on both model selection and workload characteristics. Gemma consistently provides the best trade-off between latency and stability, reinforcing the importance of using well-optimised models in CPU-based inference environments. These findings underline the necessity of jointly considering model design and serving conditions when deploying large language models in production systems.

Appendix

A. MetaLlama Summary Table

Workers	Threads	Prompt	TTFT	TPOT	Prefill Thr.	Decode Thr.	Total Time
4	10	S	5137.28	138.93	9.74	7.20	49690.85
4	10	M	24997.03	143.37	7.76	6.97	104448.80
4	10	L	69696.98	147.24	7.43	6.79	115880.68
4	20	S	5146.79	138.35	9.72	7.23	49697.22
4	20	M	25263.87	143.63	7.68	6.96	108959.75
4	20	L	68401.89	146.40	7.57	6.83	113894.44
4	30	S	5153.49	139.67	9.70	7.16	53108.07
4	30	M	25368.42	143.65	7.65	6.96	99176.81
4	30	L	68559.03	147.43	7.55	6.78	114461.97
4	40	S	5133.04	138.87	9.74	7.20	54393.29
4	40	M	25156.01	144.36	7.71	6.93	120193.68
4	40	L	68814.92	146.32	7.53	6.84	112432.56
8	10	S	5168.69	144.38	9.67	6.93	68487.97
8	10	M	25454.73	147.70	7.62	6.77	112356.36
8	10	L	68525.17	150.66	7.56	6.64	117739.45
8	20	S	5162.99	142.74	9.68	7.01	50834.29
8	20	M	25137.36	147.34	7.72	6.79	107111.66
8	20	L	69222.24	150.68	7.48	6.64	115281.54
8	30	S	5146.85	143.03	9.71	6.99	58127.51
8	30	M	25402.98	148.48	7.64	6.73	117868.18
8	30	L	68069.02	150.92	7.61	6.62	115814.12
8	40	S	5152.76	142.22	9.70	7.03	57214.11
8	40	M	25191.57	147.05	7.70	6.80	117716.63
8	40	L	68317.16	149.61	7.58	6.68	115848.35
12	10	S	5162.47	146.27	9.69	6.84	59433.19
12	10	M	25329.41	151.47	7.66	6.60	118287.52
12	10	L	68181.93	154.21	7.60	6.48	115073.61
12	20	S	5170.30	146.97	9.67	6.80	45977.53
12	20	M	25546.16	152.58	7.60	6.55	122392.04
12	20	L	68332.56	154.82	7.58	6.46	115450.08
12	30	S	5168.93	147.22	9.67	6.79	62794.79
12	30	M	25066.15	152.64	7.74	6.55	137018.11
12	30	L	68923.15	154.94	7.52	6.45	117939.48
12	40	S	5153.19	147.01	9.70	6.80	57648.16
12	40	M	25177.29	152.03	7.71	6.58	118914.84
12	40	L	68707.63	154.88	7.54	6.45	114192.71

TABLE I
MetaLlama inference performance summary.

B. TinyLlama Summary Table

Workers	Threads	Prompt	TTFT	TPOT	Prefill Thr.	Decode Thr.	Total Time
4	10	S	6711.04	33.49	42.50	29.99	16394.83
4	10	M	20999.78	38.88	31.04	25.85	39932.54
4	10	L	24112.84	47.41	34.68	21.11	36891.49
4	20	S	7114.49	32.54	36.69	31.02	21023.30
4	20	M	17965.18	37.96	35.48	26.37	42388.08
4	20	L	28109.77	43.57	32.50	23.19	41230.51
4	30	S	10238.94	35.69	38.82	28.04	23389.66
4	30	M	17517.47	43.78	38.24	22.98	38826.71
4	30	L	36593.14	52.12	30.80	19.37	52819.20
4	40	S	11485.56	35.22	38.88	28.57	28354.10
4	40	M	14366.83	41.60	40.39	24.14	44781.15
4	40	L	35815.14	42.68	30.79	23.58	46707.63
8	10	S	4816.07	40.16	45.32	25.01	16370.23
8	10	M	18881.30	54.73	31.22	18.61	66397.37
8	10	L	38495.34	51.61	29.48	19.43	56342.87
8	20	S	15022.11	39.97	34.78	25.08	30176.38
8	20	M	13292.67	47.12	37.45	21.31	49719.91
8	20	L	24805.40	48.47	32.15	20.65	41385.58
8	30	S	7218.88	37.69	41.88	26.75	19621.97
8	30	M	15923.24	45.26	34.79	22.28	43173.07
8	30	L	29534.91	48.98	33.19	20.43	46869.43
8	40	S	7721.46	36.88	39.82	27.22	21626.23
8	40	M	20647.66	44.61	39.14	22.76	45891.19
8	40	L	29978.33	50.08	32.80	20.21	46758.46
12	10	S	6659.12	41.18	41.99	24.28	28868.94
12	10	M	10007.35	47.42	44.37	21.09	45768.93
12	10	L	29656.60	55.56	32.15	18.35	44937.74
12	20	S	15405.82	41.97	37.41	23.93	30547.53
12	20	M	36408.78	51.68	29.49	19.44	74876.27
12	20	L	58659.99	55.01	24.36	18.32	74582.65
12	30	S	9983.68	40.39	40.28	24.93	25093.34
12	30	M	14099.98	44.89	38.96	22.40	45728.06
12	30	L	34136.54	51.48	30.99	19.68	47780.33
12	40	S	11256.86	39.57	36.75	25.37	24753.54
12	40	M	22333.55	48.66	34.35	20.67	55863.33
12	40	L	30469.78	49.47	32.01	20.35	47600.52

TABLE II
TINYLLAMA INFERENCE PERFORMANCE SUMMARY.

C. Gemma Summary Table

Workers	Threads	Prompt	TTFT	TPOT	Prefill Thr.	Decode Thr.	Total Time
4	10	S	1596.51	83.82	31.32	11.93	32268.94
4	10	M	9567.97	83.49	20.76	11.98	56956.99
4	10	L	26119.85	85.32	21.99	11.72	69800.19
4	20	S	1594.28	83.97	31.36	11.91	34811.88
4	20	M	9669.98	83.63	20.57	11.96	58341.80
4	20	L	26096.79	84.71	21.98	11.81	63173.58
4	30	S	1599.42	83.42	31.26	11.99	31769.94
4	30	M	9271.93	83.75	20.71	11.94	63523.25
4	30	L	25915.04	84.51	22.32	11.83	67467.07
4	40	S	1599.72	83.65	31.26	11.96	30808.47
4	40	M	9279.21	83.68	20.69	11.95	61819.47
4	40	L	25924.86	84.29	22.32	11.87	63572.54
8	10	S	1614.47	89.36	30.97	11.19	31547.11
8	10	M	9597.70	89.17	20.69	11.21	60752.04
8	10	L	26766.19	90.44	21.48	11.06	76436.90
8	20	S	1614.01	89.53	30.98	11.17	35238.44
8	20	M	9303.42	89.49	20.64	11.17	66116.21
8	20	L	25877.49	90.60	22.34	11.04	74959.80
8	30	S	1617.78	89.55	30.91	11.17	37058.58
8	30	M	9303.96	89.37	20.64	11.19	59505.19
8	30	L	25981.09	90.57	22.27	11.04	76220.43
8	40	S	1610.70	88.76	31.05	11.27	33383.93
8	40	M	9303.45	89.03	20.64	11.23	62991.33
8	40	L	25960.62	89.64	22.27	11.16	73172.98
12	10	S	1629.73	92.14	30.68	10.85	37442.44
12	10	M	9516.76	93.05	20.90	10.75	59118.70
12	10	L	26225.76	94.26	21.91	10.61	74637.40
12	20	S	1625.18	93.11	30.77	10.74	30620.56
12	20	M	9706.46	92.86	20.49	10.77	66847.49
12	20	L	26792.12	94.66	21.45	10.56	68661.92
12	30	S	1625.04	92.25	30.77	10.84	37124.66
12	30	M	9304.37	93.08	20.64	10.74	62090.56
12	30	L	25982.45	93.60	22.25	10.68	75554.49
12	40	S	1626.99	91.71	30.73	10.90	31158.67
12	40	M	9610.46	92.65	20.70	10.79	62852.76
12	40	L	26237.21	93.86	21.90	10.66	73019.66

TABLE III
GEMMA INFERENCE PERFORMANCE SUMMARY.